



**RV College of
Engineering**

Go, change the world

Unit-2

Multiprocessors & Thread-Level Parallelism

By Dr. Sandhya . S

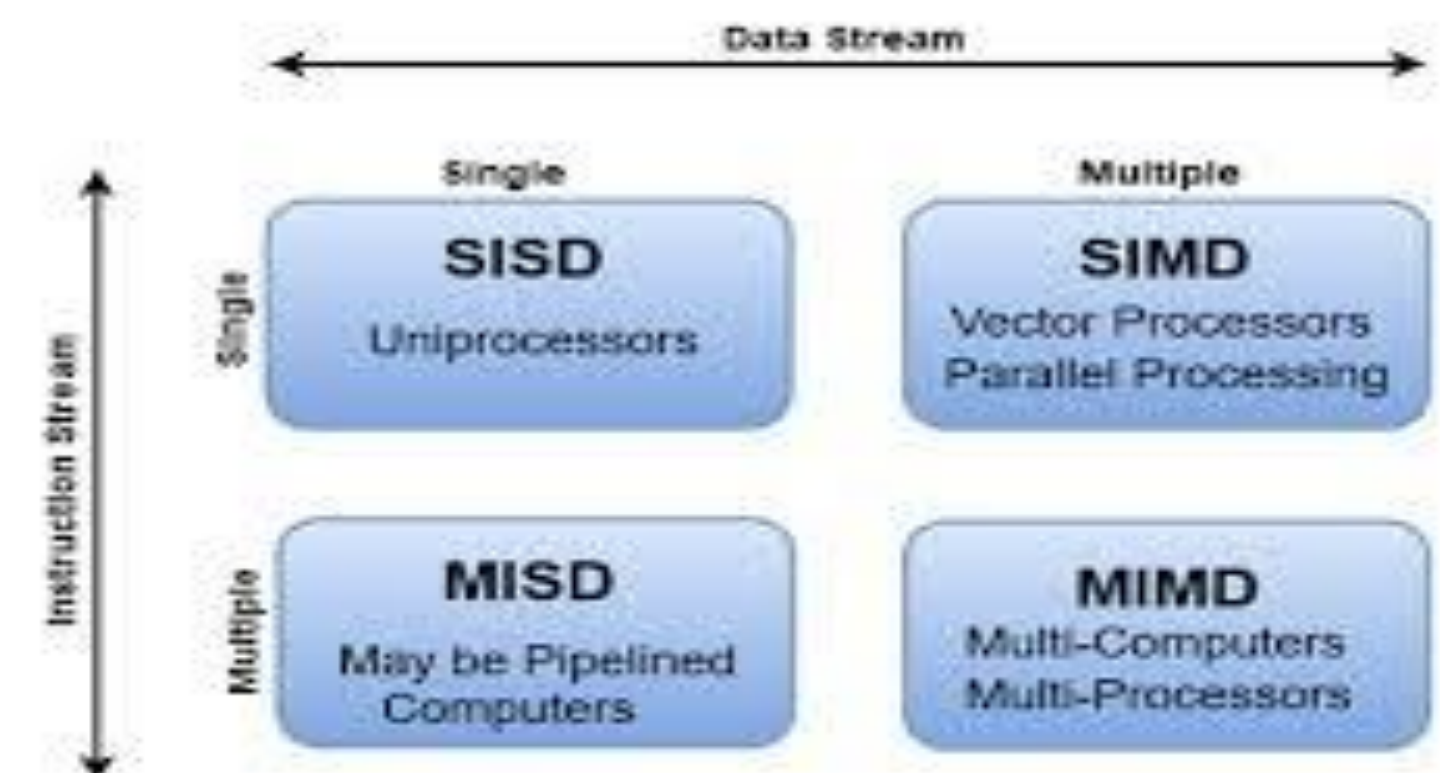
Slide Sources: Based on CA: aQA by Hennessy/Patterson.
Supplemented from various freely downloadable sources
Copyright © 2012, Elsevier Inc. All rights reserved.

Flynn's Classification of Computers:

Flynn's Taxonomy

- Single instruction stream, single data stream (SISD)
- Single instruction stream, multiple data streams (SIMD)
- Multiple instruction streams, single data stream (MISD)
- Multiple instruction streams, multiple data streams (MIMD)

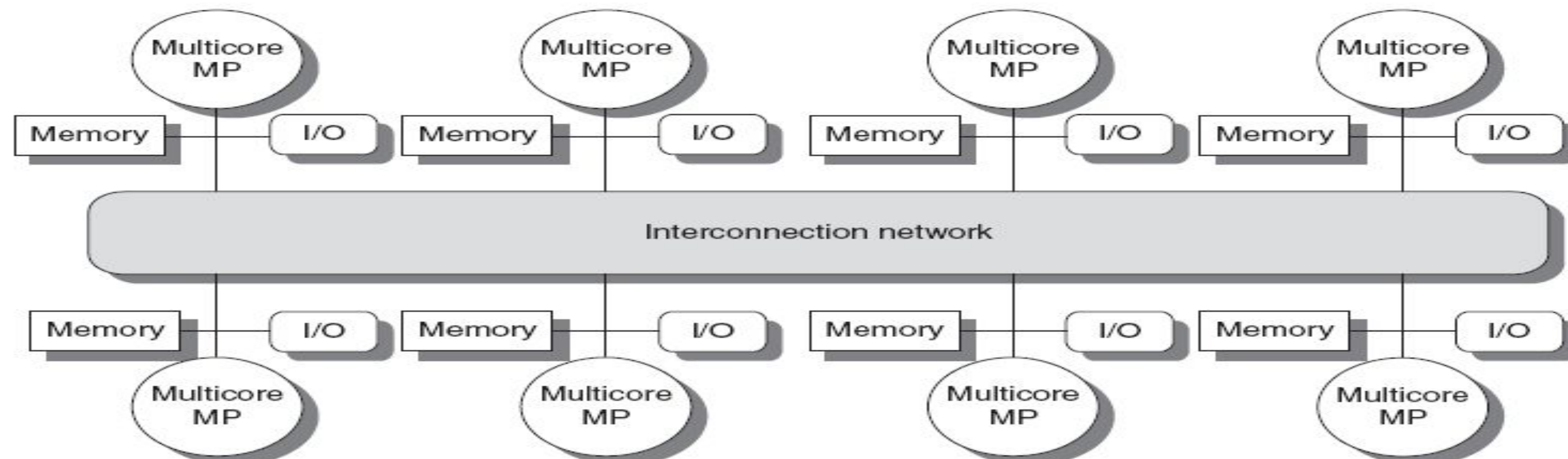
Flynn's Classification of Computers



Models for Communication

Two alternative Architectural approaches :

1. Communication through shared address space
2. Message –passing multiprocessors



- Thread-Level parallelism
 - Have multiple program counters
 - Uses MIMD model
 - Targeted for tightly-coupled shared-memory multiprocessors
- For n processors, need n threads
- Amount of computation assigned to each thread = **grain size**
 - Threads can be used for data-level parallelism, but the overheads may outweigh the benefit

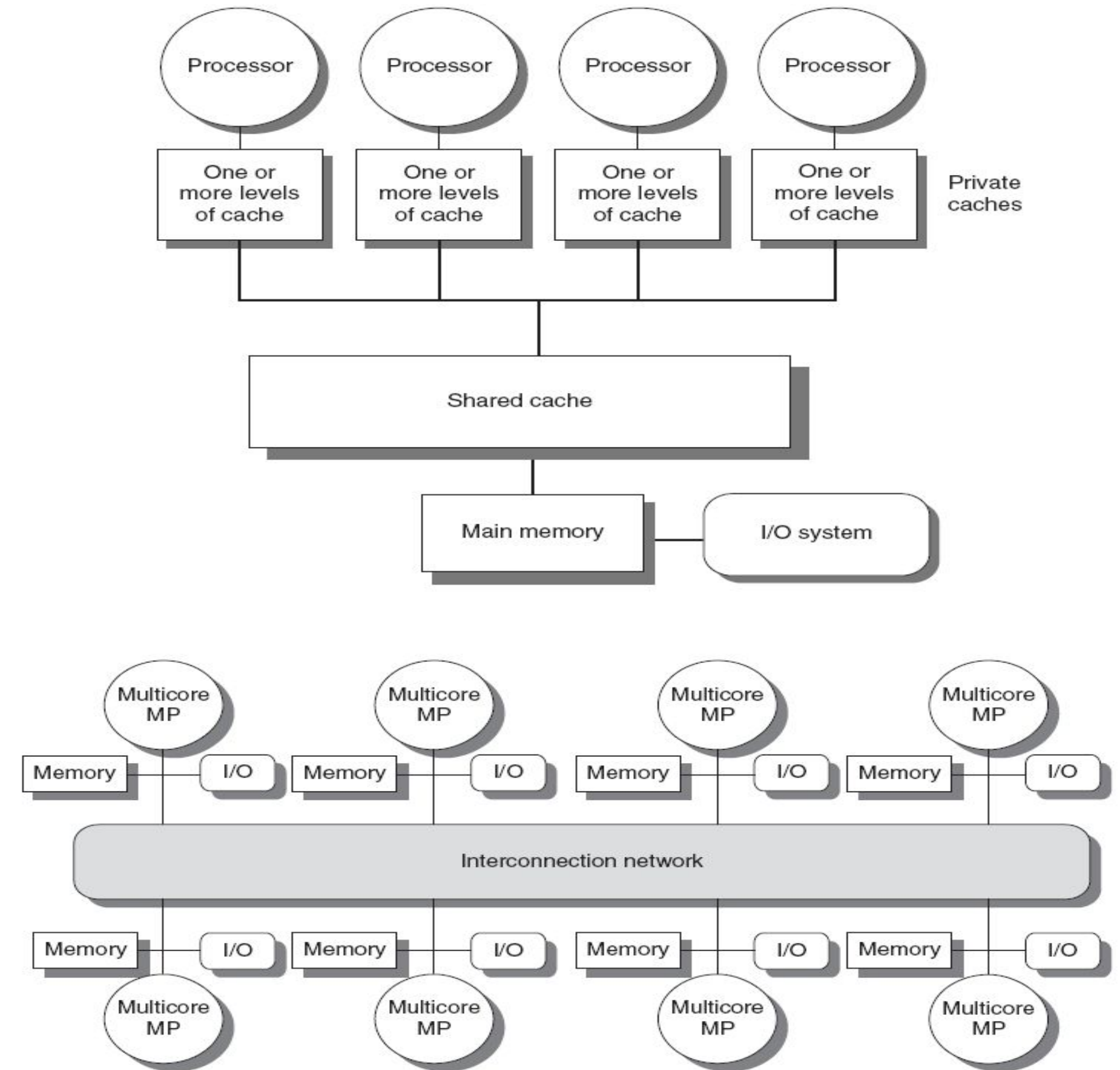
Why Use MIMDs?

1. MIMD's offer Flexibility
2. MIMD's can build on cost-performance advantages

Examples:

Clusters (Commodity Clusters & Custom Clusters)

- Symmetric multiprocessors (SMP)
 - Small number of cores
 - Share single memory with uniform memory latency
- Distributed shared memory (DSM)
 - Memory distributed among processors
 - Non-uniform memory access/latency (NUMA)
 - Processors connected via direct (switched) and non-direct (multi-hop) interconnection networks



Challenges of Parallel Processing

Two Important Challenges:

- i. The limited parallelism available in programs.
- ii. The relatively high cost of communications.

Problem1:

Suppose you want to achieve a speedup of 80 with 100 processors. What fraction of the original computation can be sequential?

Amdahl's Law is

$$\text{Speedup} = \frac{1}{\frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} + (1 - \text{Fraction}_{\text{enhanced}})}$$

Speedup is 80 and $\text{Speedup}_{\text{enhanced}} = 100$

$$80 = \frac{1}{\frac{\text{Fraction}_{\text{parallel}}}{100} + (1 - \text{Fraction}_{\text{parallel}})}$$

Problem1: Contd...

$$\text{Fraction}_{\text{parallel}} = \frac{80 - 1}{79.2}$$

- Hence, $\text{Fraction}_{\text{parallel}} = 0.9975$

Therefore the percentage of sequential computation is 0.25%

Problem2:

Suppose we have an application running on a 32-processor multiprocessor, which has a 200 ns time to handle reference to a remote memory. For this application, assume that all the references except those involving communication hit in the local memory hierarchy, which is slightly optimistic. Processors are stalled on a remote request, and the processor clock rate is 2 GHz. If the base CPI (assuming that all references hit in the cache) is 0.5, how much faster is the multiprocessor if there is no communication versus if 0.2% of the instructions involve a remote communication reference?

Problem2: Contd...

- The effective CPI for the multiprocessor with 0.2% remote references is

$$\text{CPI} = \text{Base CPI} + \text{Remote request rate} \times \text{Remote request cost}$$

$$= 0.5 + 0.2\% \times \text{Remote request cost}$$

The remote request cost is

$$\text{Remote access cost/Cycle time} = 200 \text{ ns}/0.5 \text{ ns} = 400 \text{ cycles}$$

The CPI: $\text{CPI} = 0.5 + 0.8 = 1.3$

The multiprocessor with all local references is $1.3/0.5 = 2.6$ times faster.

Cache Coherence

- SMP's support both caching of **Private Data** & **Shared Data**.

Multiprocessor Cache Coherence:

- Processors may see different values through their caches:

Time	Event	Cache contents for CPU A	Cache contents for CPU B	Memory contents for location X
0				1
1	CPU A reads X	1		1
2	CPU B reads X	1	1	1
3	CPU A stores 0 into X	0	1	0

Cache Coherence

Terms **Coherence** and **consistency** have a thin boundary of distinction.

A memory system is coherent if

1. A read by a processor P to a location X that follows a write by P to X, with no writes of X by another processor occurring between the write and the read by P, always returns the value written by P.
2. A read by a processor to location X that follows a write by another processor to X returns the written value if the read and write are sufficiently separated in time and no other writes to X occur between the two accesses.
3. Writes to the same location are *serialized*; that is, two writes to the same location by any two processors are seen in the same order by all processors.

For example, if the values 1 and then 2 are written to a location, processors can never read the value of the location as 2 and then later read it as 1.

Enforcing Coherence

- Coherent caches provide:
 - Migration*: movement of data
 - Replication*: multiple copies of data
- Cache coherence protocols
 - Directory based
 - Sharing status of each block kept in one location
 - Snooping
 - Each core tracks sharing status of each block

Snoopy Coherence Protocols

1. Write invalidate

- On write, invalidate all other copies
- Use bus itself to serialize
 - Write cannot complete until bus access is obtained

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

Fig: Write invalidation for write back caches

2. Write update/write broadcast

- On write, update all copies

Snoopy Coherence Protocols

- Can have multiple readers, but ***single writer***.
 - To *write a shared data*: an invalidate is sent to all caches which snoop and invalidate any copies
 - On *Read Miss*:
 1. *Write-Through* – memory is always up-to-date.
 2. *Write-Back* – snoop in caches to find the most recent copy.
- Locating an item when a read miss occurs
 - In write-back cache, the updated value must be sent to the requesting processor
- Cache lines marked as shared or exclusive/modified
 - Only writes to shared lines need an invalidate broadcast
 - ✓ After this, the line is marked as exclusive

Various phenomena combine to determine performance-

1. The overall cache performance
2. The misses that arise from interprocessor communication, called coherence misses,
 - i. *True sharing misses*
 - ii. *False sharing,*

Assume that words x1 and x2 are in the same cache block, which is in the shared state in the caches of both P1 and P2. Assuming the following sequence of events, identify each miss as a true sharing miss, a false sharing miss, or a hit.

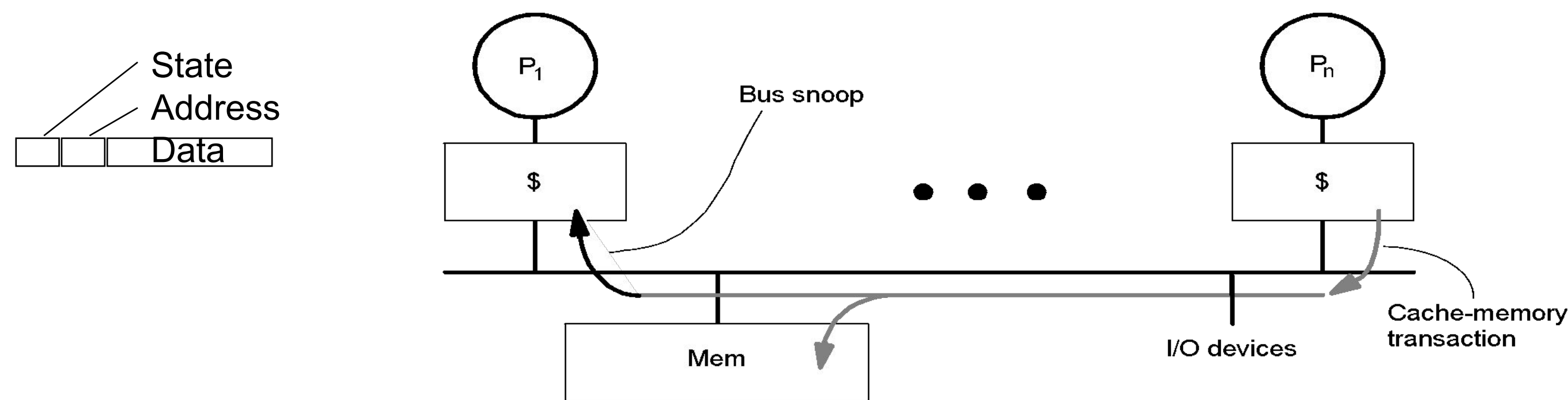
Time	P1	P2	Hit / miss
1	Write X1		true sharing miss
2		Read X2	false sharing miss
3	Write X1		hit
4		Write X2	false sharing miss
5	Read X2		True sharing miss

- Preserve Program Order: A read by processor P to location X that follows a write by P to X, with no writes of X by another processor occurring between the write and the read by P, always returns the value written by P
- Coherent view of memory: Read by a processor to location X that follows a write by another processor to X returns the written value if the read and write are sufficiently separated in time and no other writes to X occur between the two accesses
- Write serialization: 2 writes to same location by any 2 processors are seen in the same order by all processors
 - If not, a processor could keep value 1 since saw as last write
 - For example, if the values 1 and then 2 are written to a location, processors can never read the value of the location as 2 and then later read it as 1

- For now assume
- A write does not complete (and allow the next write to occur) until all processors have seen the effect of that write
- The processor does not change the order of any write with respect to any other memory access
- $\Rightarrow$ if a processor writes location A followed by location B, any processor that sees the new value of B must also see the new value of A
- These restrictions allow the processor to reorder reads, but forces the processor to finish writes in program order

- Program on multiple processors will normally have copies of the same data in several caches
 - Unlike I/O, where its rare
- Rather than trying to avoid sharing in SW, SMPs use a HW protocol to maintain coherent caches
 - Migration and Replication key to performance of shared data
- Migration - data can be moved to a local cache and used there in a transparent fashion
 - Reduces both latency to access shared data that is allocated remotely and bandwidth demand on the shared memory
- Replication – for shared data being simultaneously read, since caches make a copy of data in local cache
 - Reduces both latency of access and contention for read shared data

- Directory based — Sharing status of a block of physical memory is kept in just one location, the directory
- Snooping — Every cache with a copy of data also has a copy of sharing status of block, but no centralized state is kept
 - All caches are accessible via some broadcast medium (a bus or switch)
 - All cache controllers monitor or snoop on the medium to determine whether or not they have a copy of a block that is requested on a bus or switch access



- Cache Controller “**snoops**” all transactions on the shared medium (bus or switch)
 - relevant transaction if for a block it contains
 - take action to ensure coherence
 - invalidate, update, or supply value
 - depends on state of the block and the protocol
- Either get exclusive access before write via write invalidate or update all copies on write

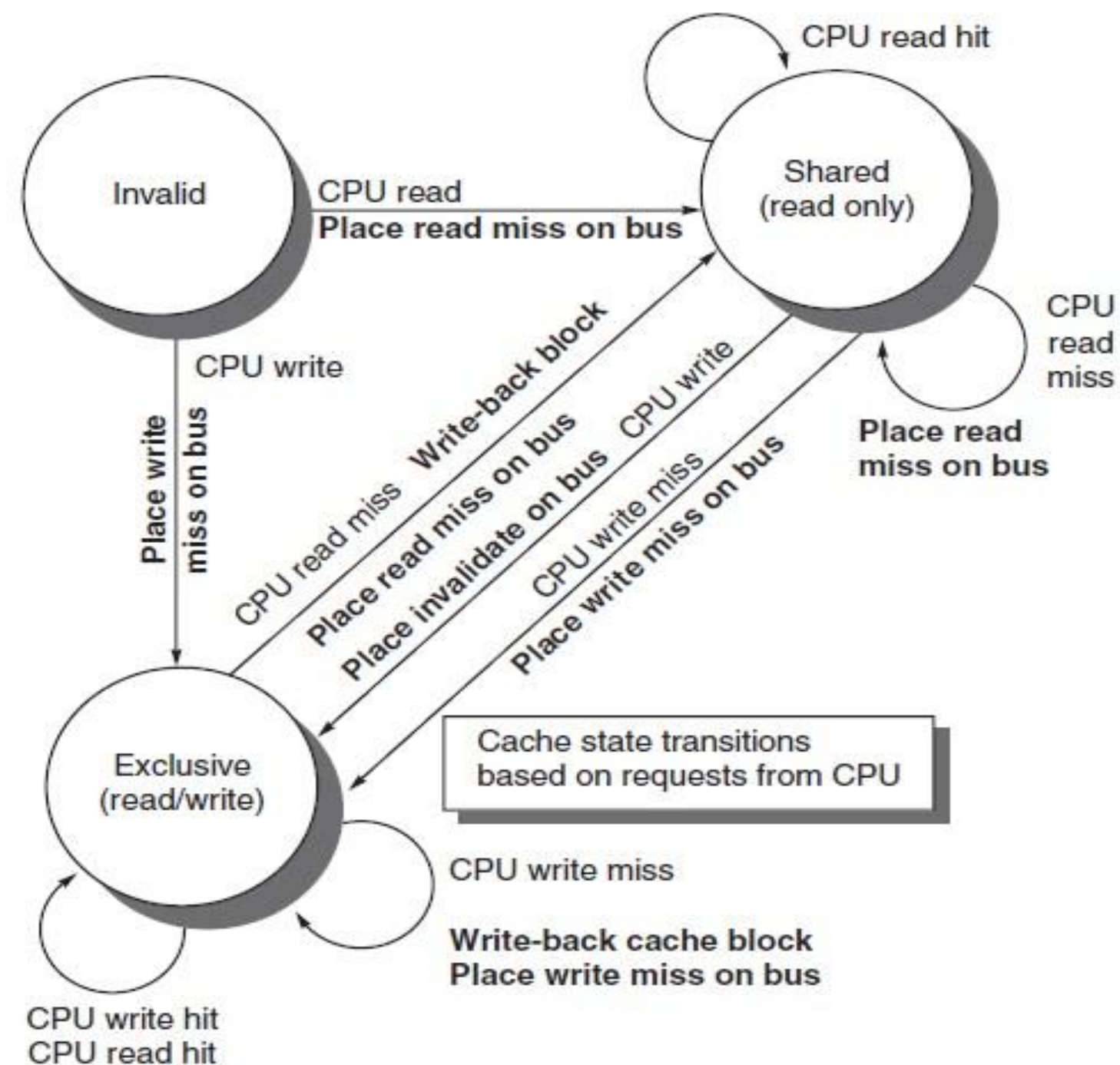
- Normal cache tags can be used for snooping
- Valid bit per block makes invalidation easy
- Read misses easy since rely on snooping
- Writes $\Rightarrow$ Need to know if know whether any other copies of the block are cached
 - No other copies $\Rightarrow$ No need to place write on bus for WB
 - Other copies $\Rightarrow$ Need to place invalidate on bus

- To track whether a cache block is shared, add extra state bit associated with each cache block, like valid bit and dirty bit
 - Write to Shared block $\Rightarrow$ Need to place invalidate on bus and mark cache block as private (if an option)
 - No further invalidations will be sent for that block
 - This processor called owner of cache block
 - Owner then changes state from shared to unshared (or exclusive)

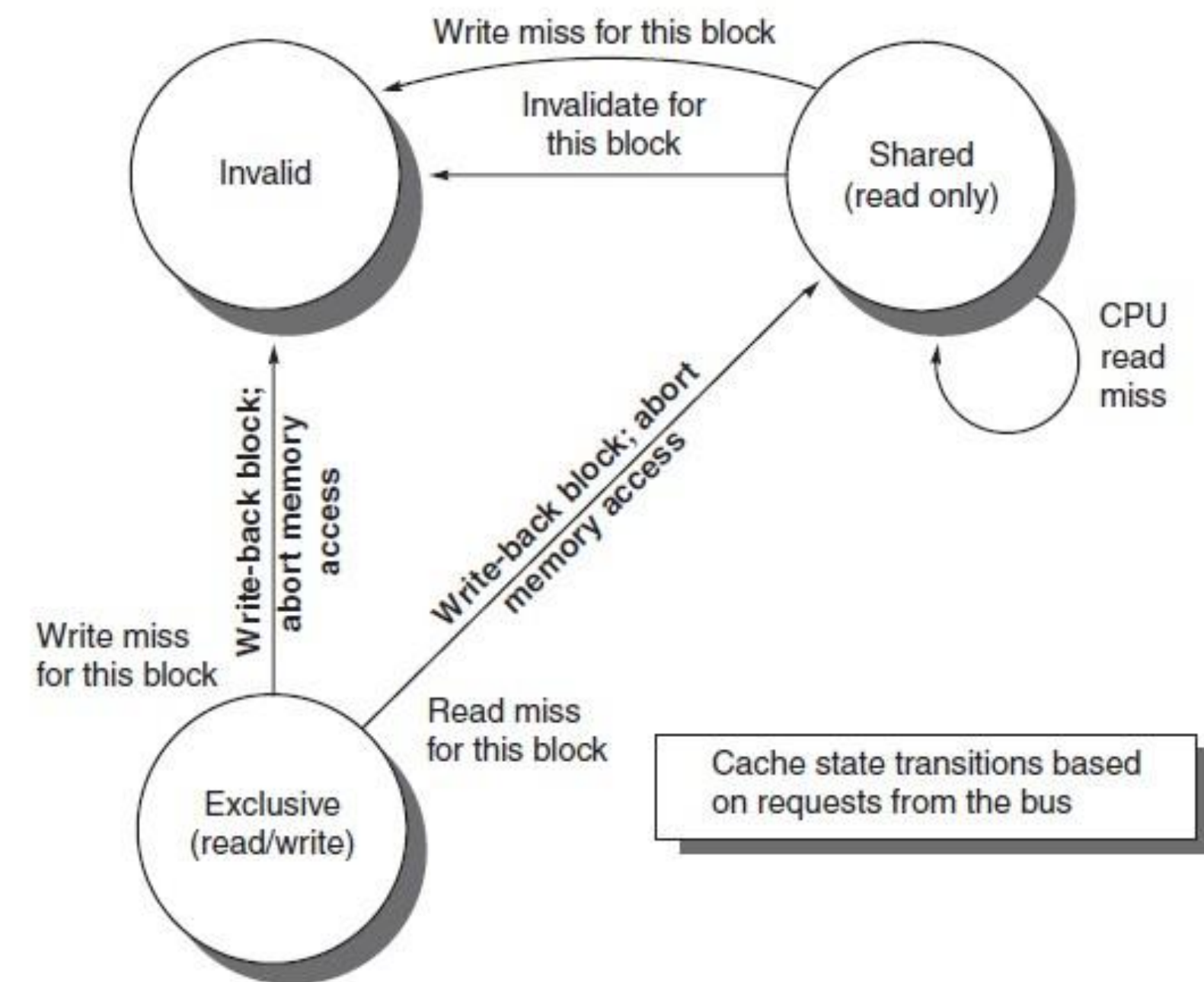
- Invalidation protocol, write-back cache
 - Snoops every address on bus
 - If it has a dirty copy of requested block, provides that block in response to the read request and aborts the memory access
- Each memory block is in one state:
 - Clean in all caches and up-to-date in memory (**Shared**)
 - OR Dirty in exactly one cache (**Exclusive**)
 - OR Not in any caches
- Each cache block is in one state (track these):
 - **Shared**: block can be read
 - OR **Exclusive**: cache has only copy, its writeable, and dirty
 - OR **Invalid**: block contains no data (in uniprocessor cache too)
- Read misses: cause all caches to snoop bus
- Writes to clean blocks are treated as misses

- **write invalidate, cache coherence protocol for a private write-back cache showing the states and state transitions for each block in the cache.**

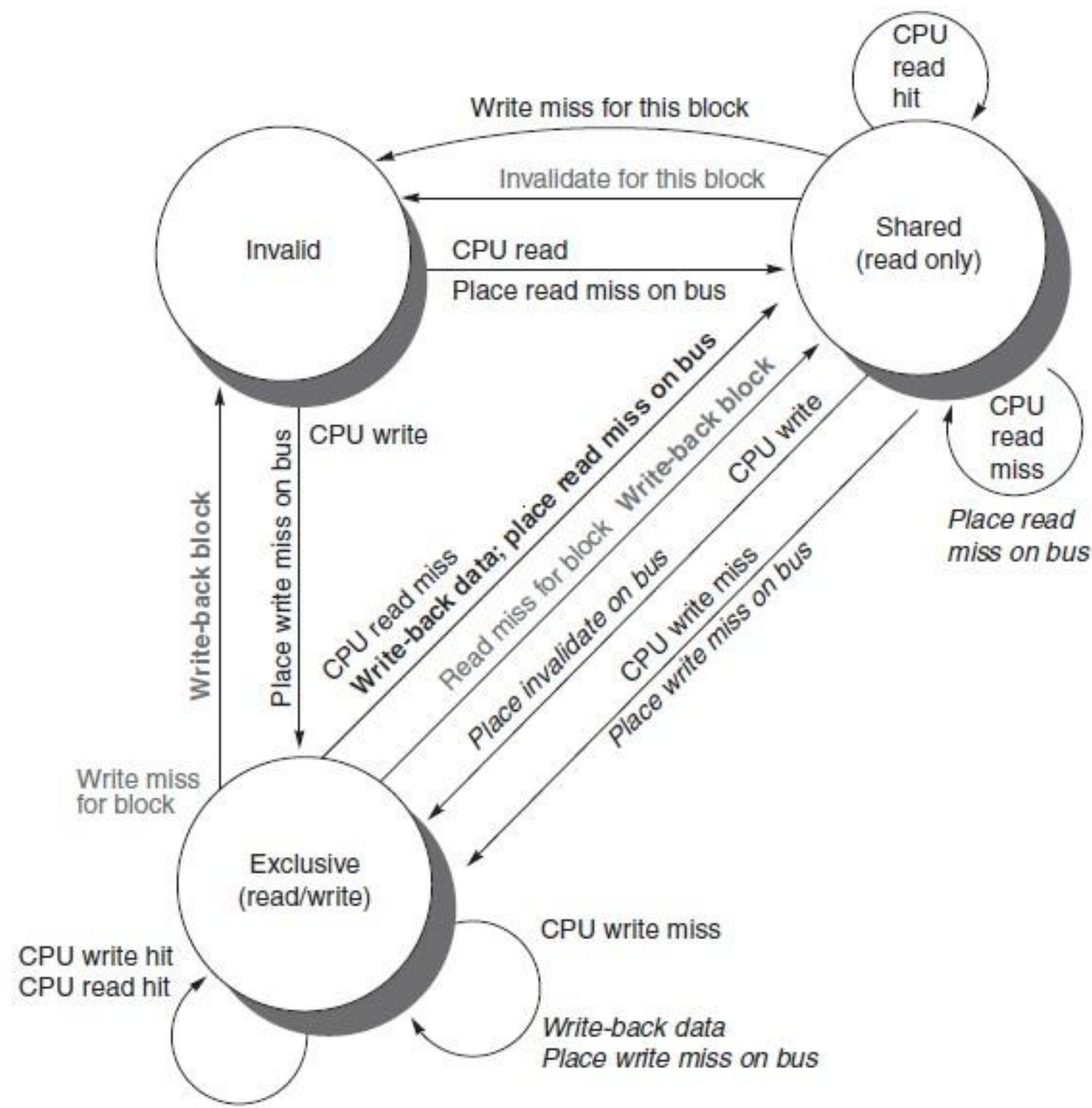
State Transitions based on actions of the Processor



State Transitions based on operations on the Bus

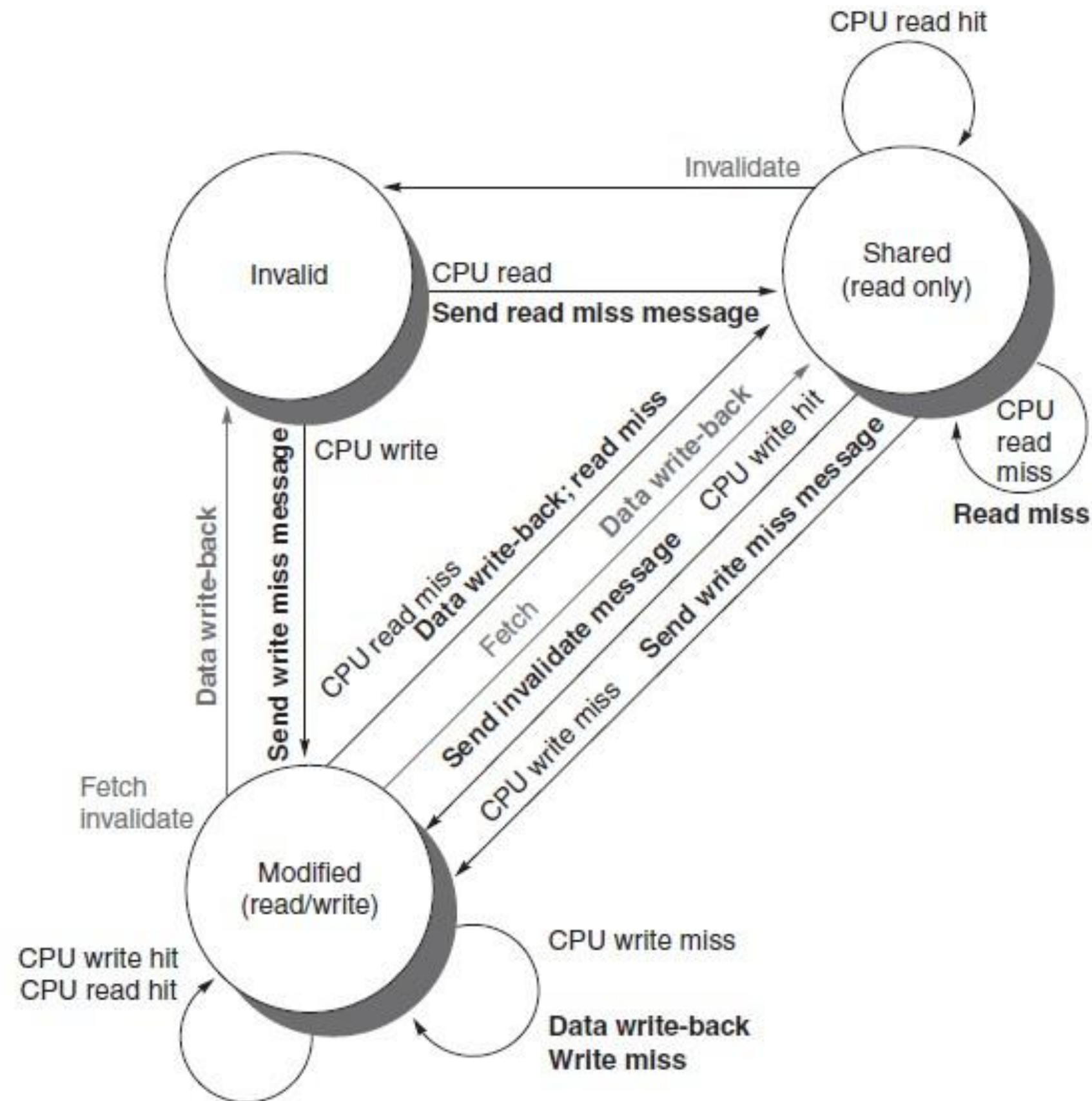


- Cache coherence state diagram with the state transitions induced by the local processor and by the bus activities – *Combined Diagram*



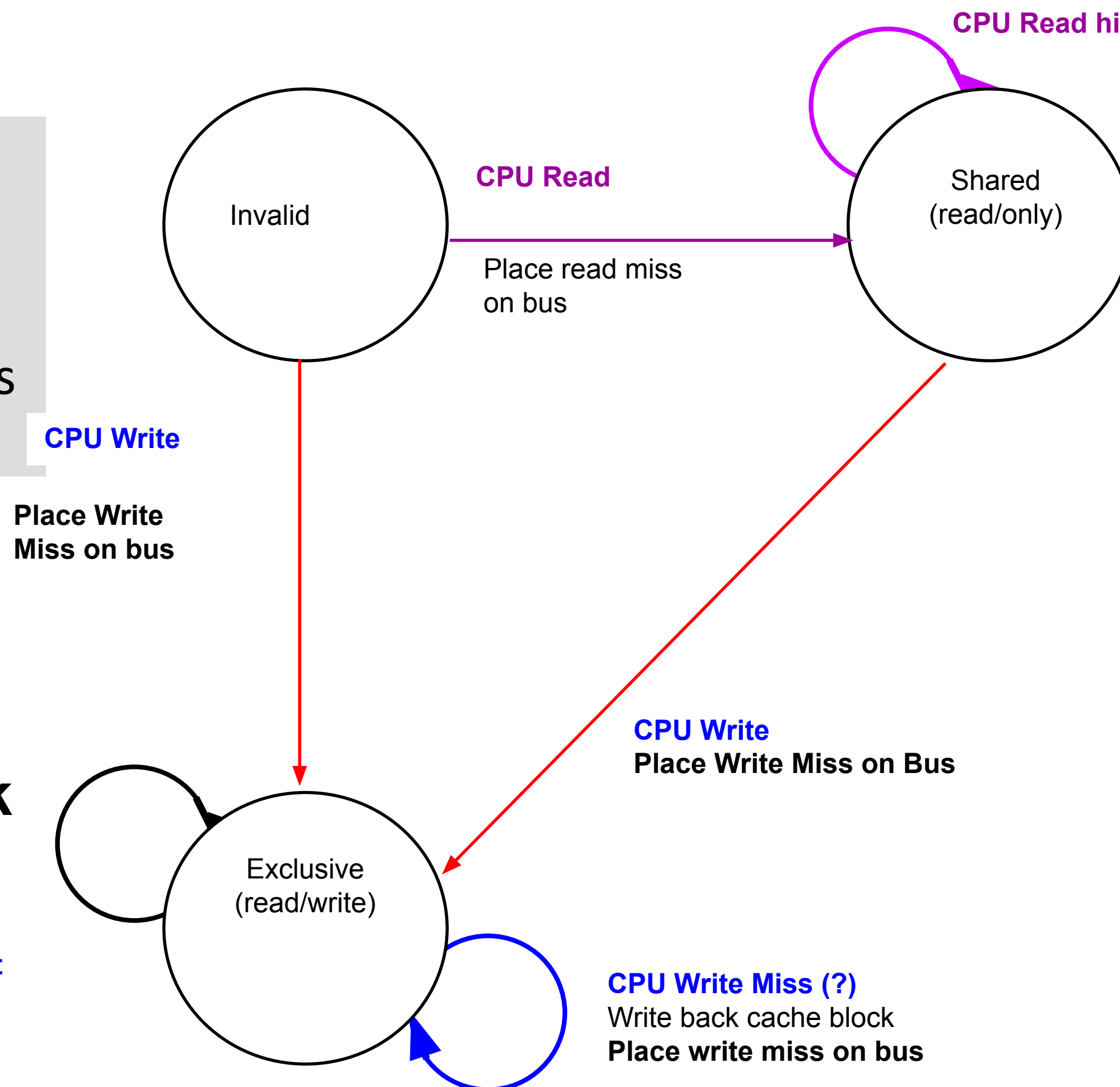
- **Cache coherence above is** a MSI (Modified, Shared, Invalid) protocol.
 - MESI* adds the state Exclusive to the basic MSI protocol to indicate when a cache block is resident only in a single cache but is clean
 - E state(cache block is resident only in a single cache but is clean), it can be written without generating any invalidates,*
 - MOESI* adds the state Owned to the MESI protocol to indicate that the associated block is owned by that cache and out-of-date in memory.
 - »adds the state Owned to indicate that the associated block is owned by that cache and out-of-date in memory*

- Cache coherence Directory



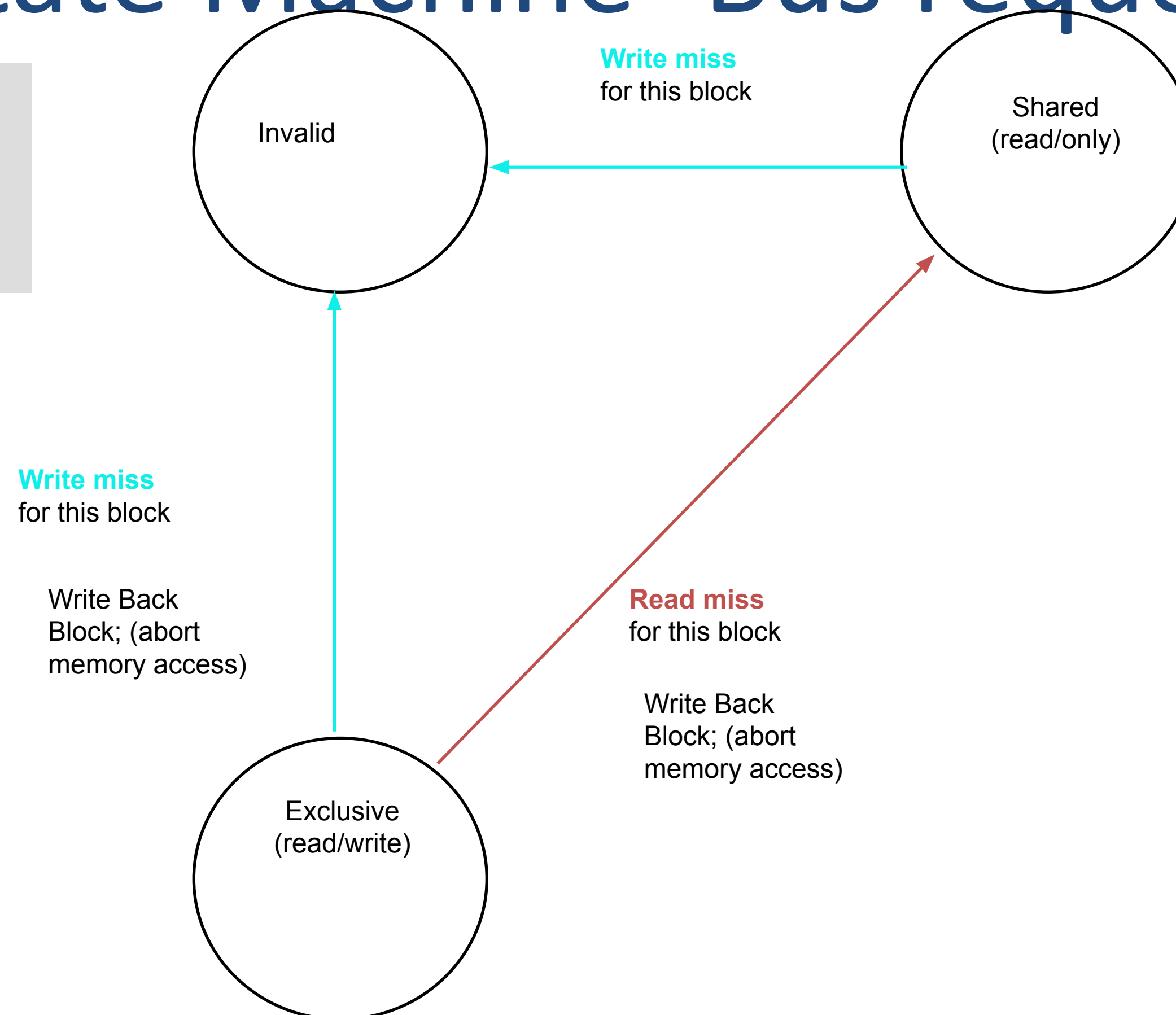
- State machine for CPU requests for each cache block
- Non-resident blocks invalid

Cache Block State



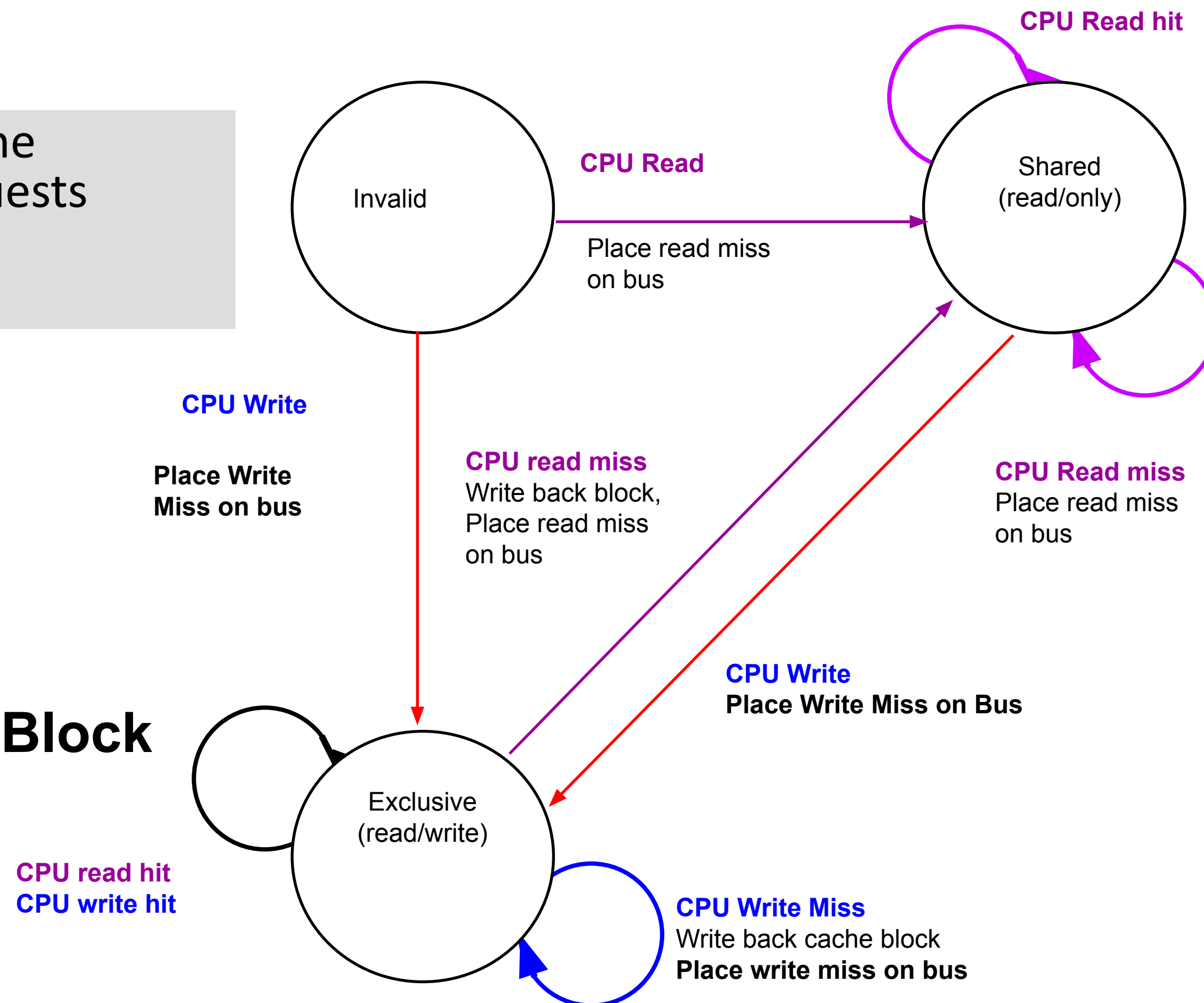
Write-Back State Machine- Bus request

- State machine for bus requests for each cache block



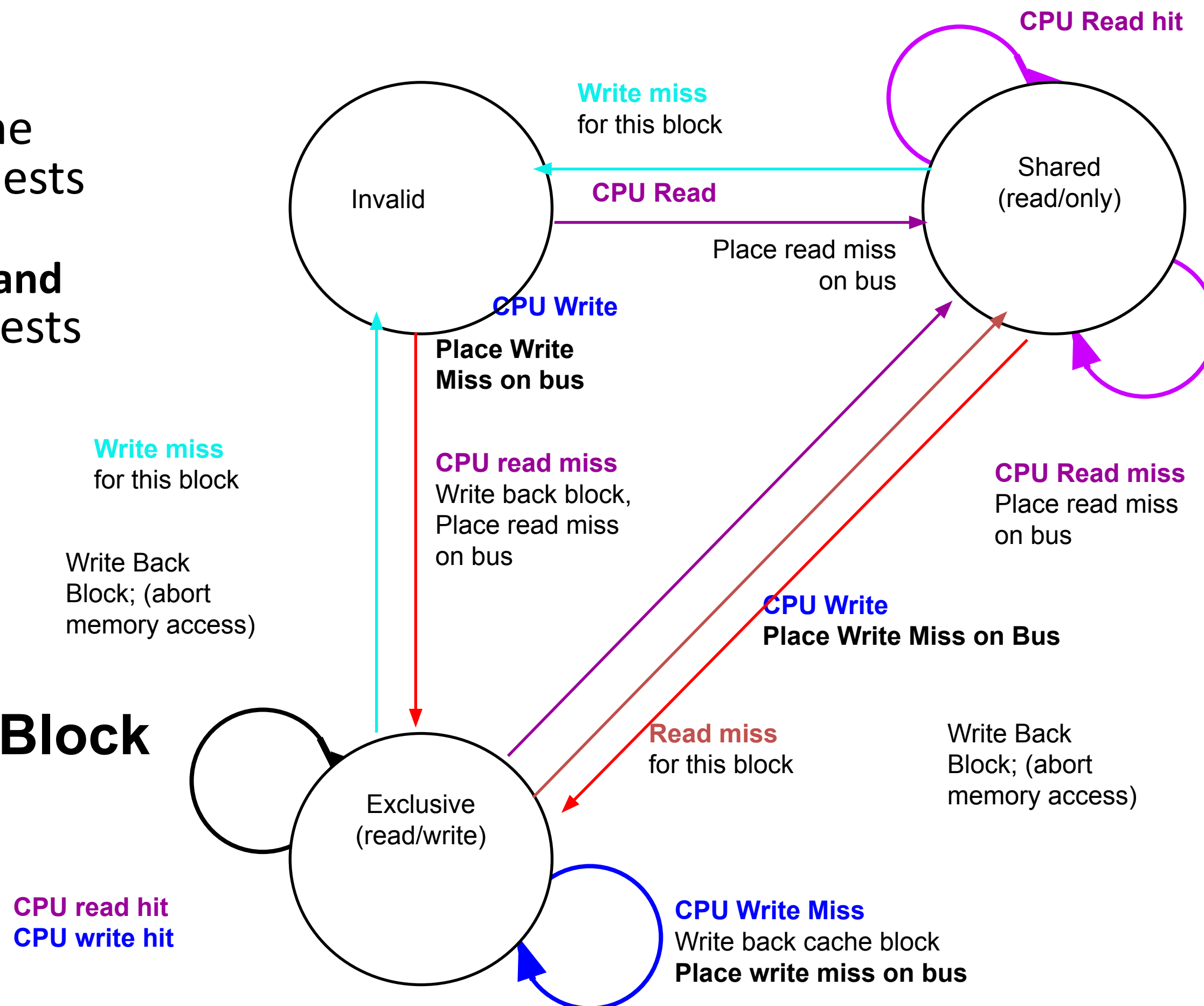
- State machine for CPU requests for each cache block

Cache Block State



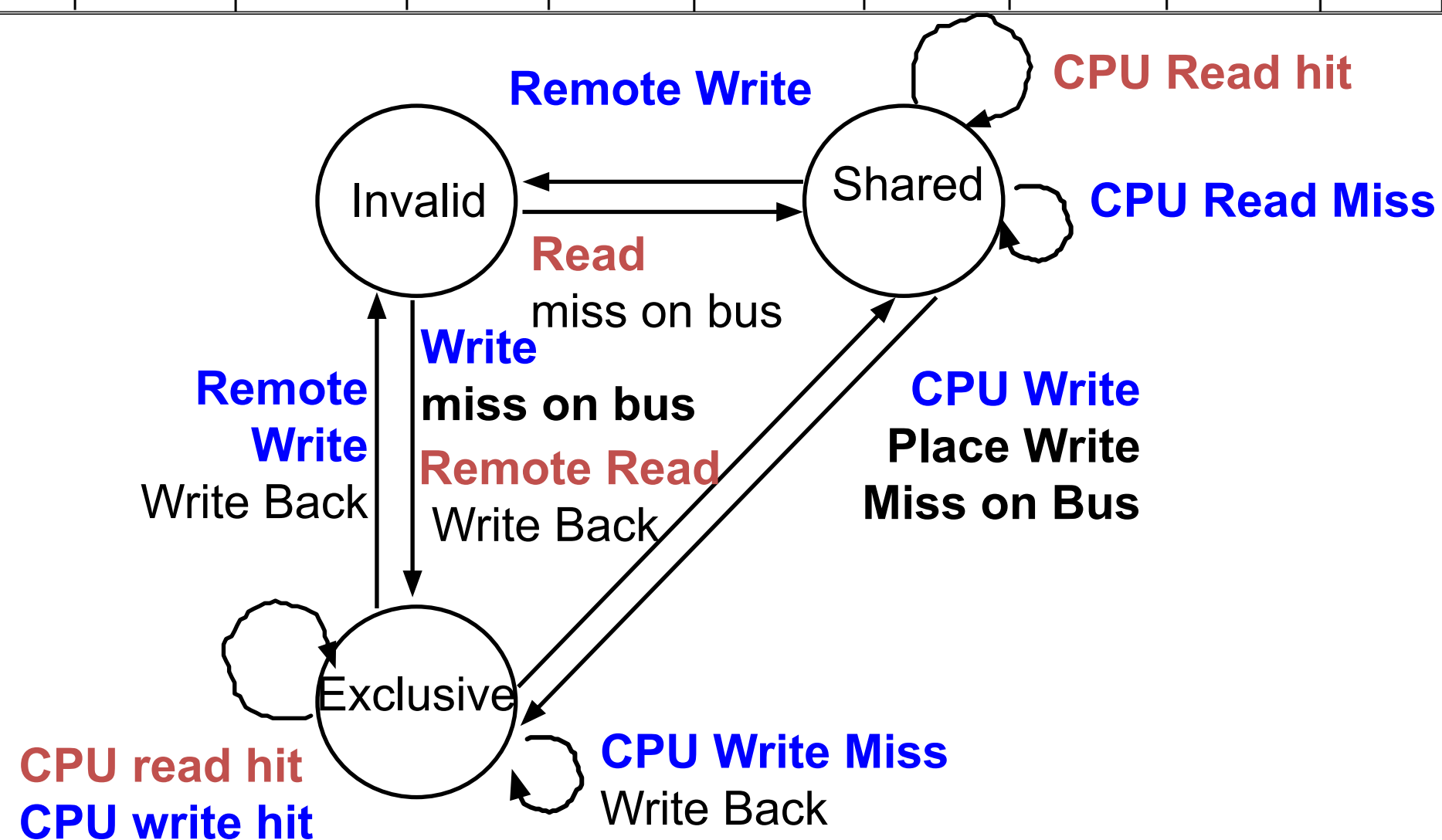
- State machine for CPU requests for each cache block and for bus requests for each cache block

Cache Block State



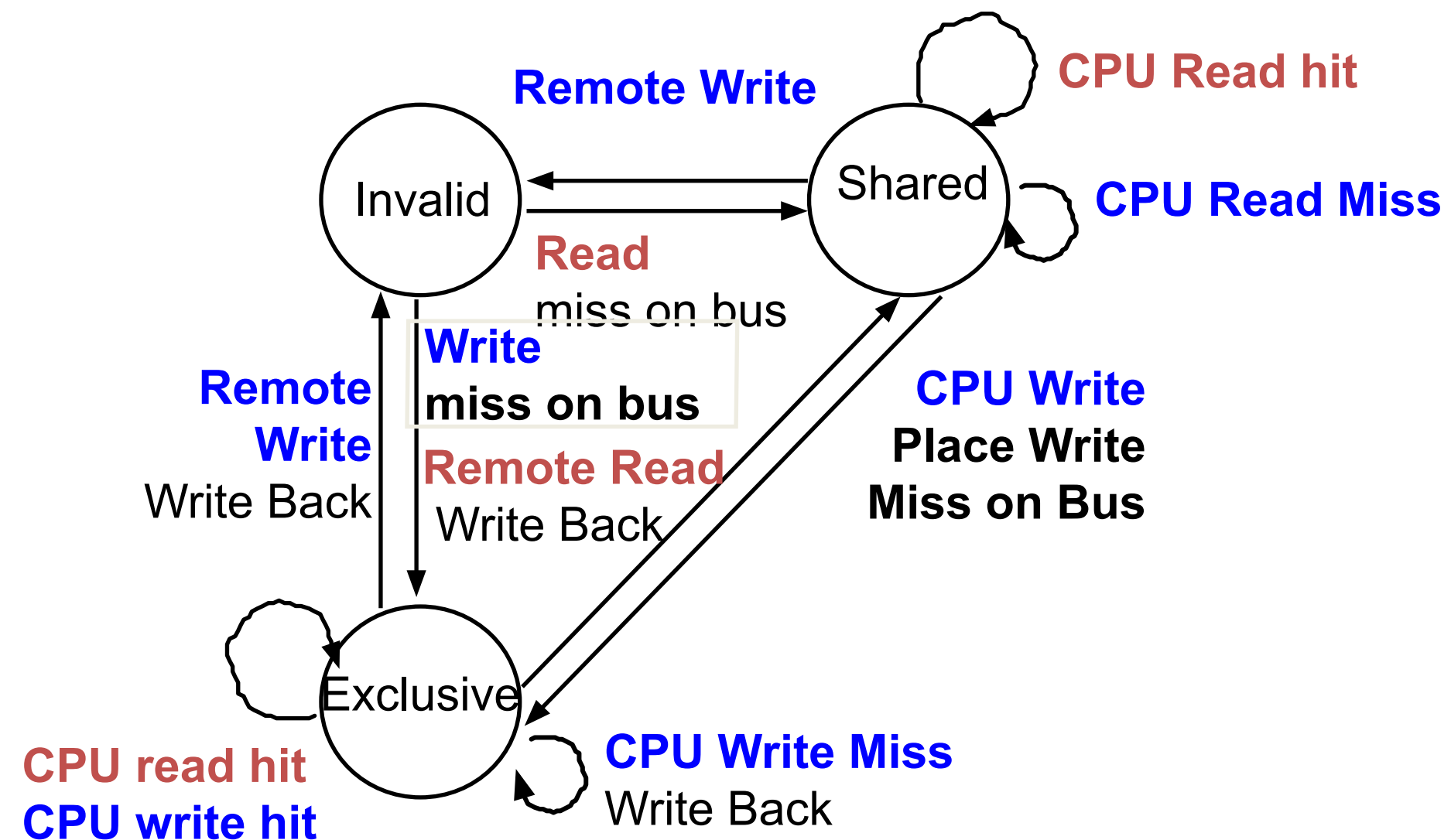
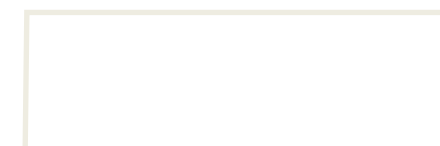
	Processor 1			Processor 2			Bus			Memory		
	<i>P1</i>			<i>P2</i>			<i>Bus</i>				<i>Memory</i>	
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	Value
P1: Write 10 to A1												
P1: Read A1												
P2: Read A1												
P2: Write 20 to A1												
P2: Write 40 to A2												

Assumes initial cache state is invalid and A1 and A2 map to same cache block, but A1 != A2



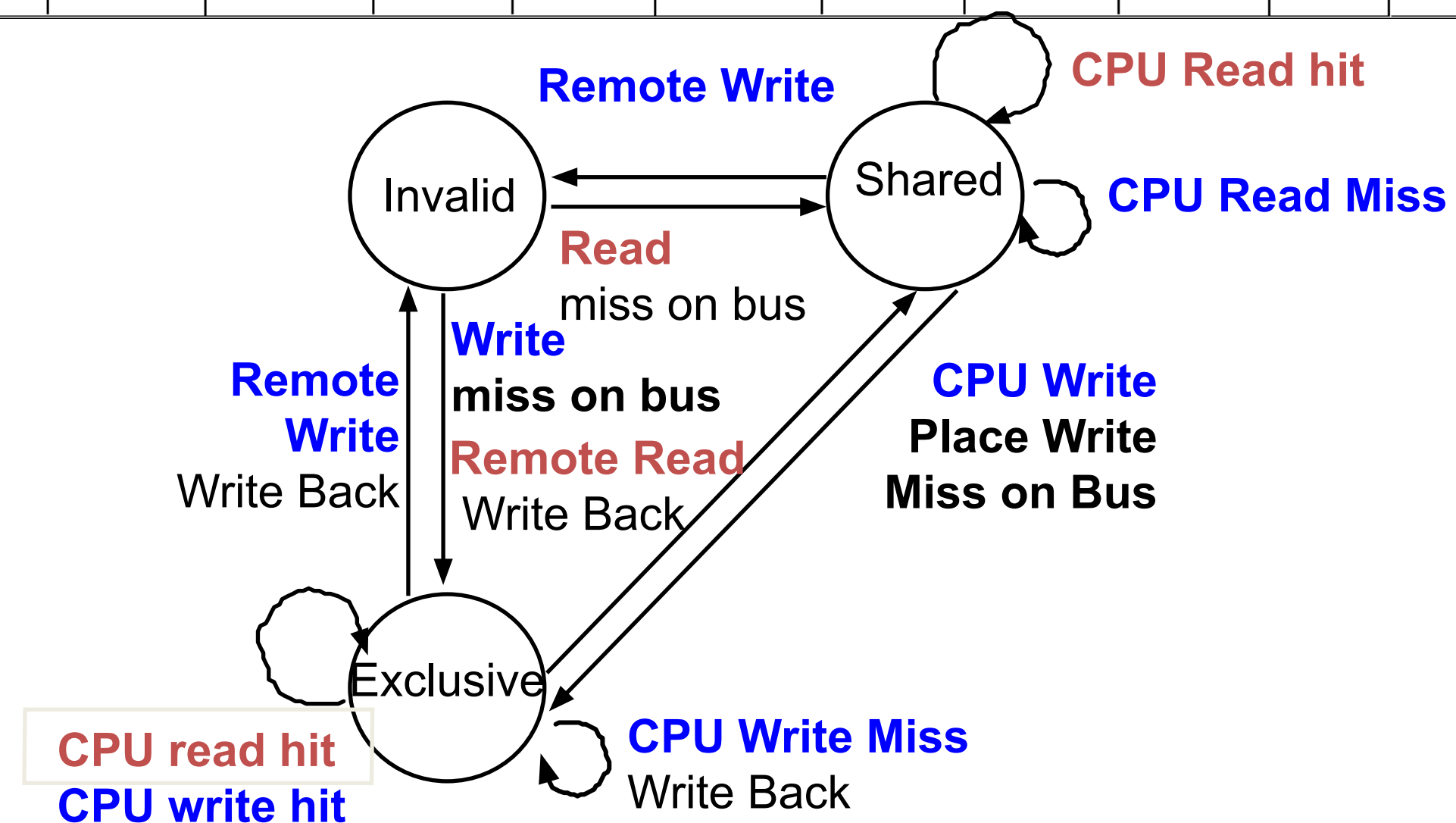
	P1			P2			Bus				Memory	
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	Value
P1: Write 10 to A1	Excl.	A1	10				WrMs	P1	A1			
P1: Read A1												
P2: Read A1												
P2: Write 20 to A1												
P2: Write 40 to A2												

Assumes initial cache state
is invalid and A1 and A2 map
to same cache block,
but A1 != A2.
Active arrow =



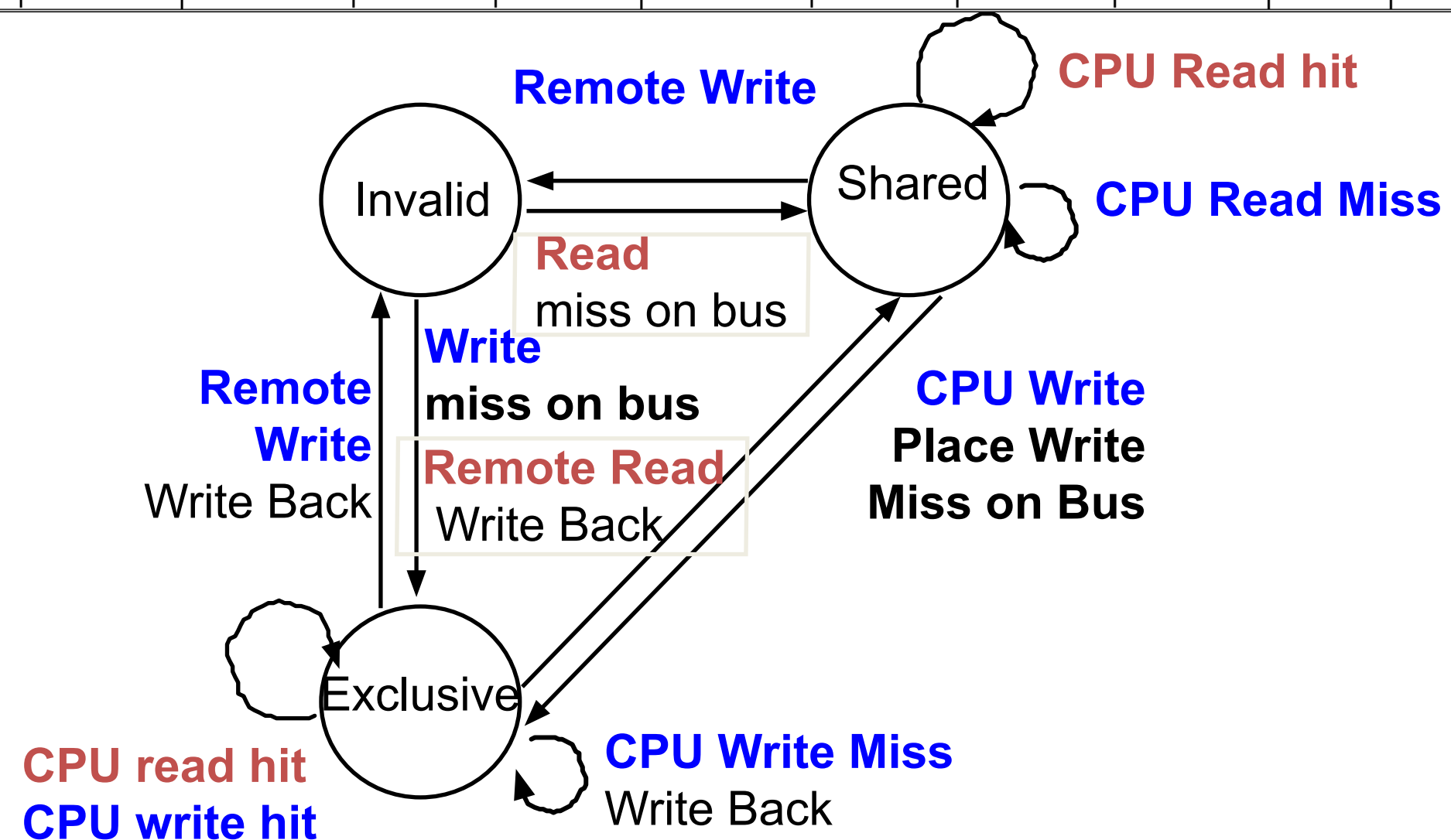
	<i>P1</i>			<i>P2</i>			<i>Bus</i>				<i>Memory</i>	
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	Value
P1: Write 10 to A1	<u>Excl.</u>	<u>A1</u>	<u>10</u>				<u>WrMs</u>	P1	A1			
P1: Read A1	Excl.	A1	10									
P2: Read A1												
P2: Write 20 to A1												
P2: Write 40 to A2												

Assumes initial cache state
is invalid and A1 and A2 map
to same cache block,
but A1 != A2



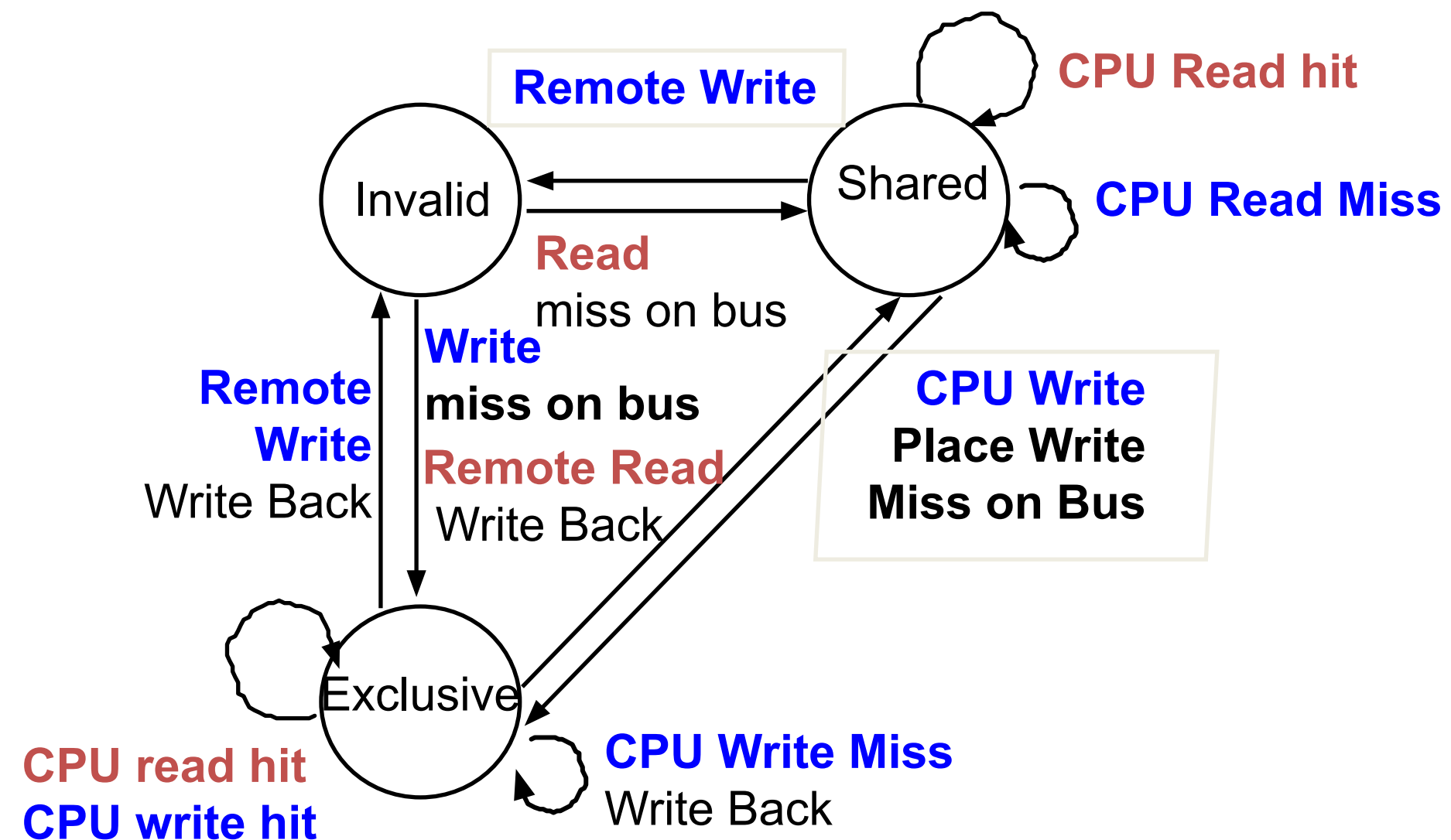
[illegible]

Assumes initial cache state is invalid and A1 and A2 map to same cache block, but $A1 \neq A2$.



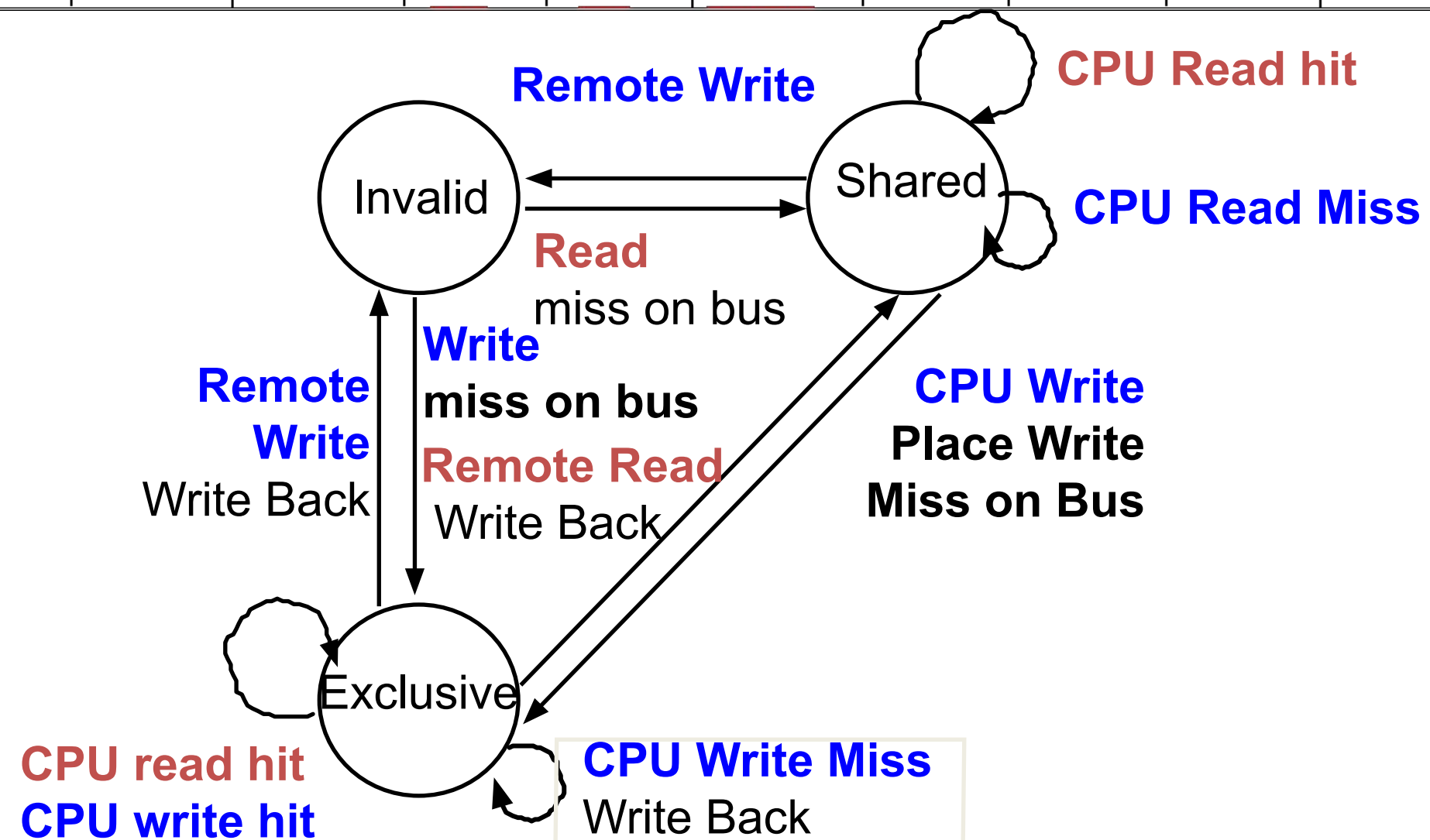
[illegible]

Assumes initial cache state is invalid and A1 and A2 map to same cache block, but $A1 \neq A2$



	P1			P2			Bus					Memory	
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	Value	
P1: Write 10 to A1	Excl.	A1	10				WrMs	P1	A1				
P1: Read A1	Excl.	A1	10								A1		
P2: Read A1				Shar.	A1		RdMs	P2	A1		A1		
	Shar.	A1	10				WrBk	P1	A1	10	A1	10	
				Shar.	A1	10	RdDa	P2	A1	10	A1	10	
P2: Write 20 to A1	Inv.			Excl.	A1	20	WrMs	P2	A1		A1	10	
P2: Write 40 to A2							WrMs	P2	A2			10	
				Excl.	A2	40	WrBk	P2	A1	20		20	

Assumes initial cache state is invalid and A1 and A2 map to same cache block, but A1 != A2



- Write Races:
 - Cannot update cache until bus is obtained
 - Otherwise, another processor may get bus first, and then write the same cache block!
 - Two step process:
 - Arbitrate for bus
 - Place miss on bus and complete operation
 - If miss occurs to block while waiting for bus, handle miss (invalidate may be needed) and then restart
 - Split transaction bus:
 - Bus transaction is not atomic:
 - can have multiple outstanding transactions for a block
 - Multiple misses can interleave,
 - allowing two caches to grab block in the Exclusive state
 - Must track and prevent multiple misses for one block
- Must support interventions and invalidations

- Multiple processors must be on bus, access to both addresses and data
- Add a few new commands to perform coherency, in addition to read and write
- Processors continuously snoop on address bus
 - If address matches tag, either invalidate or update
- Since every bus transaction checks cache tags, could interfere with CPU just to check:
 - solution 1: duplicate set of tags for L1 caches just to allow checks in parallel with CPU
 - solution 2: L2 cache already duplicate, provided L2 obeys inclusion with L1 cache
 - block size, associativity of L2 affects L1

- Bus serializes writes, getting bus ensures no one else can perform memory operation
- On a miss in a write back cache, may have the desired copy and its dirty, so must reply
- Add extra state bit to cache to determine shared or not
- Add 4th state (MESI)

Limitations in Symmetric Shared-Memory Multiprocessors and Snooping Protocols

- Single memory accommodate all CPUs
⇒ Multiple memory banks
- Bus-based multiprocessor, bus must support both coherence traffic & normal memory traffic
- ⇒ Multiple buses or interconnection networks (cross bar or small point-to-point)
- Opteron
 - Memory connected directly to each dual-core chip
 - Point-to-point connections for up to 4 chips
 - Remote memory and local memory latency are similar, allowing OS Opteron as UMA computer

- Cache performance is combination of
 - Uniprocessor cache miss traffic
 - Traffic caused by communication
 - Results in invalidations and subsequent cache misses
 - 4th C: coherence miss
 - Joins Compulsory, Capacity, Conflict

- True sharing misses arise from the communication of data through the cache coherence mechanism
 - Invalidates due to 1st write to shared block
 - Reads by another CPU of modified block in different cache
 - Miss would still occur if block size were 1 word
- False sharing misses when a block is invalidated because some word in the block, other than the one being read, is written into
 - Invalidation does not cause a new value to be communicated, but only causes an extra cache miss
 - Block is shared, but no word in block is actually shared
 - ⇒ miss would not occur if block size were 1 word

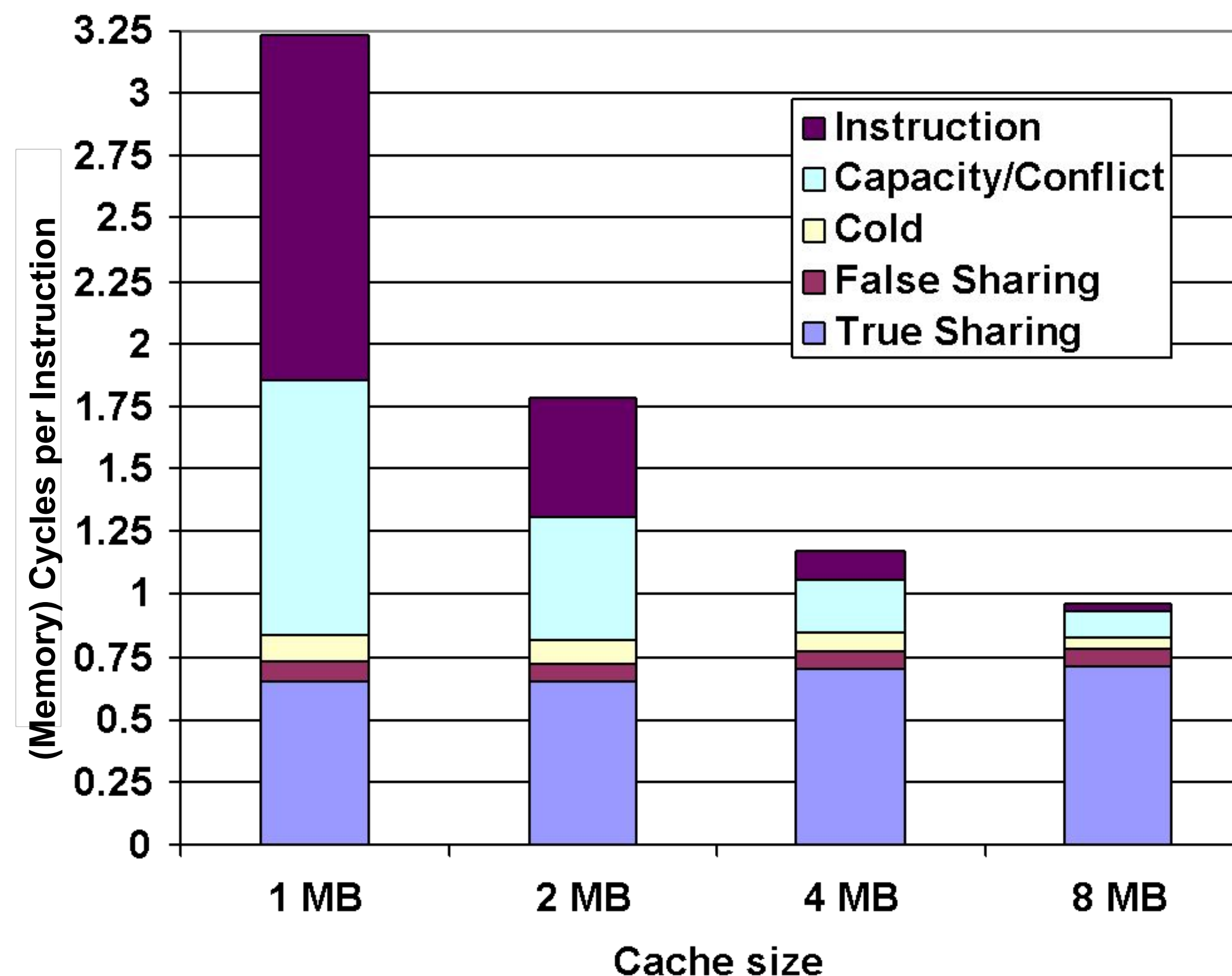
Example: True v. False Sharing v. Hit?

- Assume x1 and x2 in same cache block.
P1 and P2 both read x1 and x2 before.

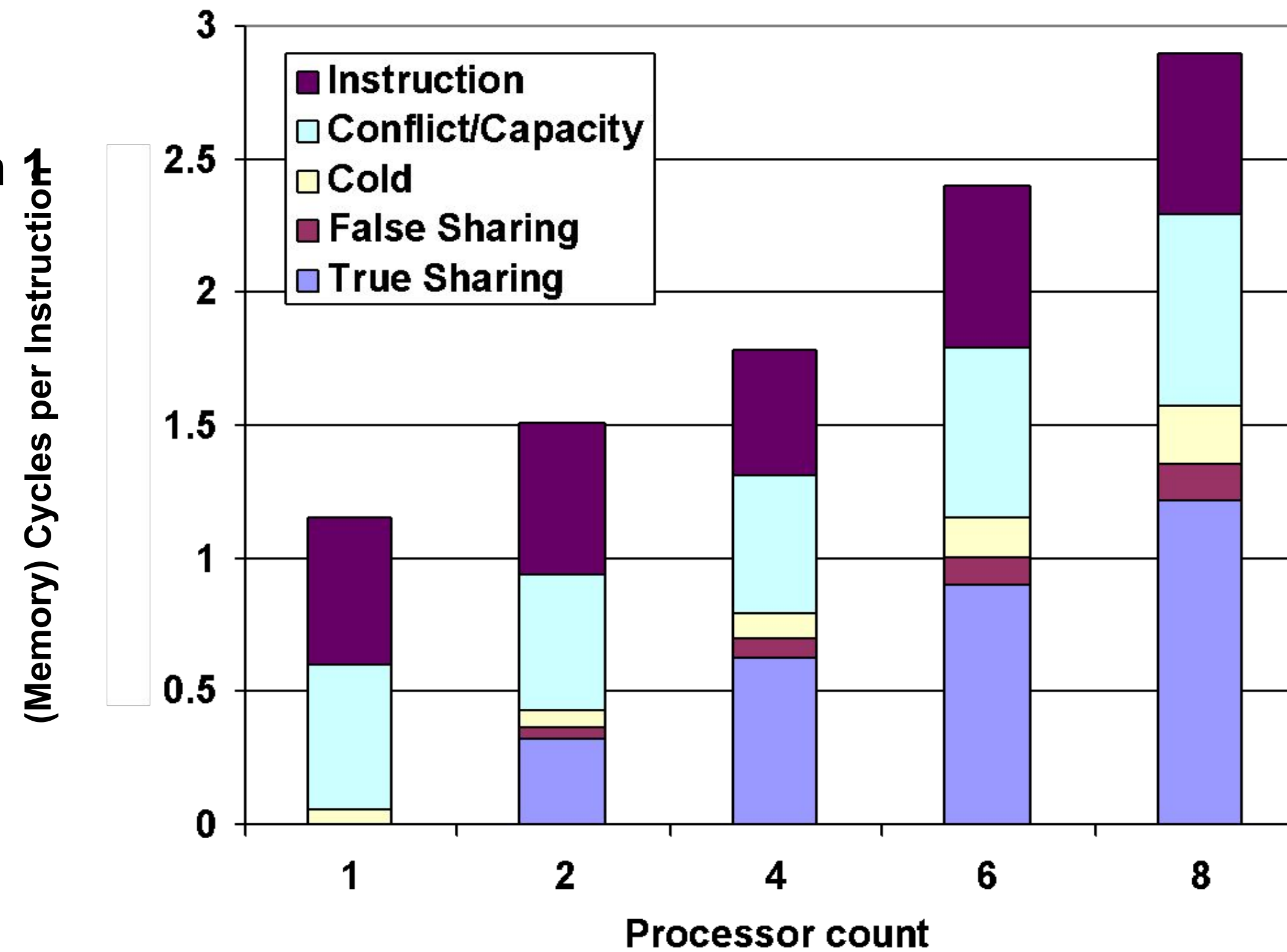
Time	P1	P2	True, False, Hit? Why?
1	Write x1		True miss; invalidate x1 in P2
2		Read x2	False miss; x1 irrelevant to P2
3	Write x1		False miss; x1 irrelevant to P2
4		Write x2	False miss; x1 irrelevant to P2
5	Read x2		True miss; invalidate x2 in P1

Engine

- True sharing and false sharing unchanged going from 1 MB to 8 MB (L3 cache)
- Uniprocessor cache misses improve with cache size increase (Instruction, Capacity/Conflict, Compulsory)



- True sharing, false sharing increase going from 1 to 8 CPUs



Sequential consistency

- The most straightforward model for memory consistency is called *sequential consistency*.
- Sequential consistency requires that the result of any execution be the same as if the memory accesses executed by each processor were kept in order and the accesses among different processors were arbitrarily interleaved.
- Sequential consistency eliminates the possibility of some nonobvious execution in the previous example because the assignments must be completed before the if statements are initiated.

Implementing sequential consistency

- The simplest way to implement sequential consistency is to require a processor to *delay the completion of any memory access until all the invalidations caused by that access are completed*. Of course, it is equally effective to delay the next memory access until the previous one is completed.
- In our example, we must delay the read of A or B i.e ($A == 0$ or $B == 0$) until the previous write has completed ($B = 1$ or $A = 1$). Under sequential consistency, **we cannot, for example, simply place the write in a write buffer and continue with the read.**

Disadvantages of Sequential consistency

- Sequential Consistency reduces potential performance. Especially in a multiprocessor with a large number of processors or long interconnect delays.

WHY?

- Since each memory access is not completed until all the invalidations on that location is completed it takes a longer time.
- In order to provide better performance there are two different routes:
 1. We can use implementations that can preserve sequential consistency but use latency hiding techniques to reduce the penalty.
 2. Developing less restrictive memory consistency models that allow for faster hardware.

The Programmer's View

- Although the sequential consistency model has a performance disadvantage, from the viewpoint of the programmer it has the advantage of simplicity.
- The challenge is to develop a programming model that is simple to explain and yet allows a high-performance implementation.
- One such programming model that allows us to have a more efficient implementation is to assume that programs are *synchronized*.
- A program is synchronized if all access to shared data are ordered by synchronization operations.
- A data reference is ordered by a synchronization operation if, in every possible execution, a write of a variable by one processor and an access (either a read or a write) of that variable by another processor are separated by a pair of synchronization operations,
- one executed after the write by the writing processor and one executed before the access by the second processor.

Data Races

- Cases where variables may be updated without ordering by synchronization are called *data races* because the execution outcome depends on the relative speed of the processors, and like races in hardware design, the outcome is unpredictable, which leads to another name for synchronized programs: *data-race-free*.
- As a simple example, consider a variable being read and updated by two different processors. Each processor surrounds the read and update with a lock and an unlock, both to ensure mutual exclusion for the update and to ensure that the read is consistent. Clearly, every write is now separated from a read by the other processor by a pair of synchronization operations: one unlock (after the write) and one lock (before the read). Of course, if two processors are writing a variable with no intervening reads, then the writes must also be separated by synchronization operations.

Problems with unsynchronized programs

- If the accesses were unsynchronized, the behavior of the program would likely be unpredictable because the speed of execution would determine which processor won a data race and thus affect the results of the program. Even with sequential consistency, reasoning about such programs is very difficult.
- Programmers could attempt to guarantee ordering by constructing their own synchronization mechanisms, but this is extremely tricky, can lead to buggy programs.
- They may not be supported architecturally, meaning that they may not work in future generations of the multiprocessor.

Relaxed Consistency Models: The Basics

- The key idea in relaxed consistency models is to allow reads and writes to **complete out of order**, but to use synchronization operations to enforce ordering, so that a synchronized program behaves as if the processor were sequentially consistent.
- There are a variety of relaxed models that are classified according to what read and write orderings they relax. We specify the orderings by a set of rules of the form

$X \rightarrow Y$, meaning that operation X must complete before operation Y is done.

- Sequential consistency requires maintaining all four possible orderings: $R \rightarrow W$, $R \rightarrow R$, $W \rightarrow R$, and $W \rightarrow W$. The relaxed models are defined by which of these four sets of orderings they relax:

Relaxed Consistency Model

- 1) Relaxing the $W \rightarrow R$ ordering yields a model known as *total store ordering* or ***processor consistency***. Because this ordering retains ordering among writes, many programs that operate under sequential consistency operate under this model
- 2) Relaxing the $W \rightarrow W$ ordering yields a model known as ***partial store order***.
- 3) Relaxing the $R \rightarrow W$ and $R \rightarrow R$ orderings yields a variety of models including *weak ordering*, the PowerPC consistency model, and *release consistency*, depending on the details of the ordering restrictions and how synchronization operations enforce ordering.

What is Synchronization?

Special form of communication in which control information is exchanged instead of data between communicating processes residing in the same or different processors.

Why Synchronize?

Need to know when it is safe for different processes to use shared data

Synchronization enforces

- Correct sequencing of processors in a multiprocessor environment
- Ensures mutually exclusive access to shared writable data

For smaller multiprocessors

- Synchronization mechanisms are typically built with user-level software routines that rely on hardware-supplied synchronization instructions.
- low-contention situations, the key hardware capability is an uninterruptible instruction or instruction sequence capable of atomically retrieving and changing a value.

For large scale multiprocessors

- In larger-scale multiprocessors or high-contention situations, synchronization can become a performance bottleneck
- because contention introduces additional delays and because latency is potentially greater in such a multiprocessor.

Basic Hardware Primitives

Uninterruptible Instruction to Fetch and Update Memory

Techniques for building synchronization operations

Atomic exchange:

Interchange a value in a register for a value in memory

Used to build a simple lock where

0 => synchronization variable/Lock is free

1 => synchronization variable/Lock is locked and unavailable

- Set register to 1 & swap
- New value in register determines success in getting lock 0 if you succeeded in setting the lock (you were first)
1 if other processor had already claimed access
- Key is that exchange operation is indivisible; two simultaneous exchanges will be ordered by the write serialization mechanism. Therefore the race is broken- exactly one processor will perform exchange returning a '0' and other processors will return '1' when they do the exchange

Other atomic primitives used to implement Synchronization

- **Test-and-set**: tests a value and sets it if the value passes the test
0 => synchronization variable is free
1 => synchronization variable is locked and unavailable
Test-and-set tests memory address corresponding to lock if it is '0' If so it sets the lock to '1'
- **Fetch-and-increment**: it returns the value of a memory location and atomically increments it
0 => synchronization variable is free
1 => synchronization variable is locked and unavailable
Fetch-and-increment fetches contents of memory address corresponding to lock and also set it to '1'. If lock variable was '0', processor gets lock, else if lock variable was '1' already incrementing does not matter

- Implementing atomic memory operation is difficult since
Hard to have read & write in 1 instruction
Also complicates implementation of coherence since the hardware cannot allow any other operation between the read and write and yet must not deadlock
: use 2 instructions instead
- **Load linked** (or load locked) + **store conditional**
 - Load linked returns the initial value
 - Store conditional returns 1 if it succeeds (no other store to same memory location since preceding load) and 0 otherwise
 - Value returned from store conditional used to deduce whether the pair of instructions executed as atomic instruction

- Instruction pair is effectively atomic if all other operations executed by any other processor occurred before or after the pair
- If content of memory location specified by the load linked are changed before the store conditional to the same memory address occurs or if there is a context switch between the two instructions then store conditional fails and returns a '0'
- Else store conditional returns '1'

- Since load linked returns initial value of the synchronization variable and store conditional returns 1 only if it succeeds ,the following instruction sequence shows atomic swap with LL & SC on the memory location specified by R1:

try: movR3,R4 ; load exchange value

ll R2,0(R1) ; load linked

sc R3,0(R1) ; store conditional

beqz R3,try ; branch store fails (R3 = 0)

movR4,R2 ; put load value in R4

At the end of code sequence contents of R4 and ML specified by R1
has been atomically exchanged

- Other synchronization primitives can also be built from LL and SC
- Example doing fetch & increment with LL & SC:

try: **ll** R2,0(R1) ; load linked

daddui R2,R2,#1 ; increment (OK if reg-reg)

sc R2,0(R1) ; store conditional

beqz R2,try ; branch store fails (R2 = 0)

Implementation of SC instruction

- Keep track of address specified by the LL instruction in a register called **Link Register**
- If interrupt occurs or if the cache block matching the address in the link register is invalidated, then the link register is cleared
- SC instruction simply checks that its address matches with that in the link register
- If so, SC succeeds otherwise it fails

Note:

Only reg-reg instructions permitted b/w LL & SC

No. of instructions between LL and SC as small as possible else probability of context switch or other processor can access the ML

Synchronization—Operation Using

Once we have atomic operation, use coherence mechanism of a multiprocessor to implement spin locks.

Spin locks:

locks processor continuously tries to acquire, spinning around a loop trying until it succeeds

Spin locks are used when

- Lock is expected to be held for short time
- Process of locking to be of low latency when lock is available

User Level Synchronization—Operation Using this Primitive

If there was no cache coherence, simple implementation of synchronization operation is

- Keep lock variables in memory
- Processor tries continuously to acquire lock using atomic operation like exchange
- Tests whether exchange returned lock as free
- To release the lock the processor simply store '0' in the lock

Code to obtain lock on a lock variable whose address is in R1 using atomic exchange is as shown

User Level Synchronization Operation in Multi Processors with Cache Coherency

What about MP with cache coherency?

We can cache the locks using coherence mechanism to maintain the lock values

Advantage of caching locks

- Spin on local cache copy to avoid full memory latency i.e can avoid a global memory access on each attempt to acquire the lock
- Likely to get cache hits for such variables because of locality of access- time to acquire the lock is reduced

User Level Synchronization Operation in Multi Processors with Cache Coherency

Problems:

- exchange includes a write, which invalidates all other copies; this generates considerable bus traffic
- most of these writes will lead to write misses since each processor is trying to obtain the lock in an exclusive state

Solution: Modify the spin lock procedure Start by simply repeatedly reading the local copy of lock variable

until it successfully sees that the lock is available

Then attempt to acquire the lock using exchange (“test-and-set”):

User Level Synchronization Operation in Multi Processors with Cache Coherency

Go, change the world

Procedure to acquire lock in this case would be:

- A processor keeps reading and checking its local copy till value of lock indicates it is unlocked/available
- Processor then races against all other processors to see who can lock the variable first
- All processors use a swap instruction that reads the old value and stores a '1' in the lock variable
- Single winner will see a '0' and the losers will see a '1' placed by the winner
- Winning processor executes code after the lock and when finished stores '0' into the lock variable to release the lock
- The race to acquire the lock starts all over again

The code sequence to perform this spin lock is as shown

```
lockit: ld R2,0(R1) ;load lock var
        bnez R2,lockit ;not free=>spin
        daddui R2,R0,#1 ;load locked value
        exch R2,0(R1) ;atomic exchange
        bnez R2,lockit ;already locked?
```

```
unlock: store location, #0 /* free the lock (write 0) */
```

User Level Synchronization Operation in Multi-Processors with Cache Coherency

to, change the world

How does this “spin-lock” scheme use the cache-coherence mechanisms?

- **Figure shows the processor and bus or directory operations for multiple**
- **processes trying to lock a variable using an atomic swap.**
- **Once the processor with the lock stores a 0 into the lock, all other caches are invalidated and must fetch the new value to update their copy of the lock.**
- **One such cache gets the copy of the unlocked value (0) first and performs the swap.**
- **When the cache miss of other processors is satisfied, they find that the variable is already locked,**
- **so they must return to testing and spinning.**

User Level Synchronization Operation in Multi Processors with Cache Coherency

Same spin lock procedure can also be implemented
using the load linked and store conditional instructions

```
lockit: ll    R2,0(R1)      ;load lock var
        bnez  R2,lockit     ;not free=>spin
        daddui R2,R0,#1     ;load locked value
        sc    R2,0(R1)      ;store conditional
        beqz  R2,lockit     ;already locked?
```

First branch forms the spin loop

Second branch resolves races when two processors see
the lock available simultaneously

- “End” of uniprocessors speedup => Multiprocessors
- Parallelism challenges: % parallalizable, long latency to remote memory
- Centralized vs. distributed memory
 - Small MP vs. lower latency, larger BW for Larger MP
- Message Passing vs. Shared Address
 - Uniform access time vs. Non-uniform access time
- Snooping cache over shared medium for smaller MP by invalidating other cached copies on write
- Sharing cached data $\Rightarrow$ Coherence (values returned by a read), Consistency (when a written value will be returned by a read)
- Shared medium serializes writes
 - $\Rightarrow$ Write consistency